Introduction

Linear regression generally have the form of $Y_i = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$
There are several ways to find the coefficients of the regression:

1. Linear Algebra: $\hat{\theta} = (X^T X)^{-1} X^T Y$ (When X is invertible)
2. Gradient Descent: In this case, we need to write out the loss function and try to minimize the loss.

$$F(x) \text{ = Loss Function = MSE = } \frac{1}{n} \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2$$

In this part of the assignment, we will be using the second way to implement this linear regression model. More details about the model's implementation can be found in corresponding lectures.

ATTENTION: THERE ARE A TOTAL OF 4 QUESTIONS THAT NEED YOUR ANSWERS

Import necessary packages

You'll be implement your model in `LinearRegression.py` which should be put under the same directory as the location of `Linear_Regression.ipynb`. Since we have enabled `autoreload`, you only need to import these packages once. You don't need to restart the kernel of this notebook nor rerun the next cell even if you change your implementation for `LinearRegression.py` in the meantime.

A suggestion for better productivity if you never used jupyter notebook + python script together: you can split your screen into left and right parts, and have your left part displaying this notebook and have your right part displaying your `LinearRegression.py`

In [10]:
```
# Please do not change this code block
%load_ext autoreload
%autoreload 2

# import numpy, pandas, pyplot for arrays, dataframes, and visualizations
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# import sklearn model to validate our custom model
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Please make sure that your `LinearRegression.py` is under the same folder as this
from LinearRegression import Linear_Regression
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

Experiment 1: Perfect Data

In this part, we generate a dataset with a perfect linear relationship to test our model's performance. Here, we use the equation: $y = 5x + 10$ to generate our dataset.

In [11]:
```
X = np.array([np.arange(1, 1000, 5)]).T
y = np.array((5 * X)).flatten() +  10
f'x = {X[:5].flatten()}, y = {y[:5]} for the first 5 values'
```

Out[11]:
```
'x = [ 1  6 11 16 21], y = [ 15  40  65  90 115] for the first 5 values'
```

First, let's try to fit our model without any normalization (note: the below cell block could take significant amount of time to complete)

In [14]:
```
%%time
reg = Linear_Regression(num_iter = 10000000)
reg.fit(X,y)
print(f'\nNumber of total iterations: {len(reg.loss)} \nBest Loss: {min(reg.loss)}'
```

```
Start Training
----------------------
Iteration: 0
Coef: [[0.29939337]
 [0.96733965]]
Loss: 647671023168439.5
----------------------
Iteration: 1000000
Coef: [[5.00880501]
 [4.15265479]]
Loss: 1717.3264508741206
----------------------
Iteration: 2000000
Coef: [[5.0057028 ]
 [6.21183582]]
Loss: 720.7562335244493
----------------------
Iteration: 3000000
Coef: [[5.00369469]
 [7.54586287]]
Loss: 302.50221819825146
----------------------
Iteration: 4000000
Coef: [[5.00239416]
 [8.41010363]]
Loss: 126.9613259851133
----------------------
Iteration: 5000000
Coef: [[5.00155106]
 [8.96999617]]
Loss: 53.2857406611383
----------------------
Iteration: 6000000
Coef: [[5.00100437]
 [9.33271863]]
Loss: 22.36375663810404
----------------------
Iteration: 7000000
Coef: [[5.00065096]
 [9.56770626]]
Loss: 9.386194129120653
----------------------
Iteration: 8000000
Coef: [[5.00042184]
 [9.7199413 ]]
Loss: 3.939427403811304
----------------------
Iteration: 9000000
Coef: [[5.00027336]
 [9.81856582]]
Loss: 1.6533938742375351
----------------------
End Training

Number of total iterations: 10000000
Best Loss: 0.6939118672542067
```

```
CPU times: total: 4min 29s
Wall time: 4min 51s
```

Then, let's try to fit our model with min-max normalization

In [15]:
```
%%time
reg_norm = Linear_Regression(num_iter = 10000000, normalize=True)
reg_norm.fit(X,y)
print(f'\nNumber of total iterations: {len(reg_norm.loss)} \nBest Loss: {min(reg_no
```

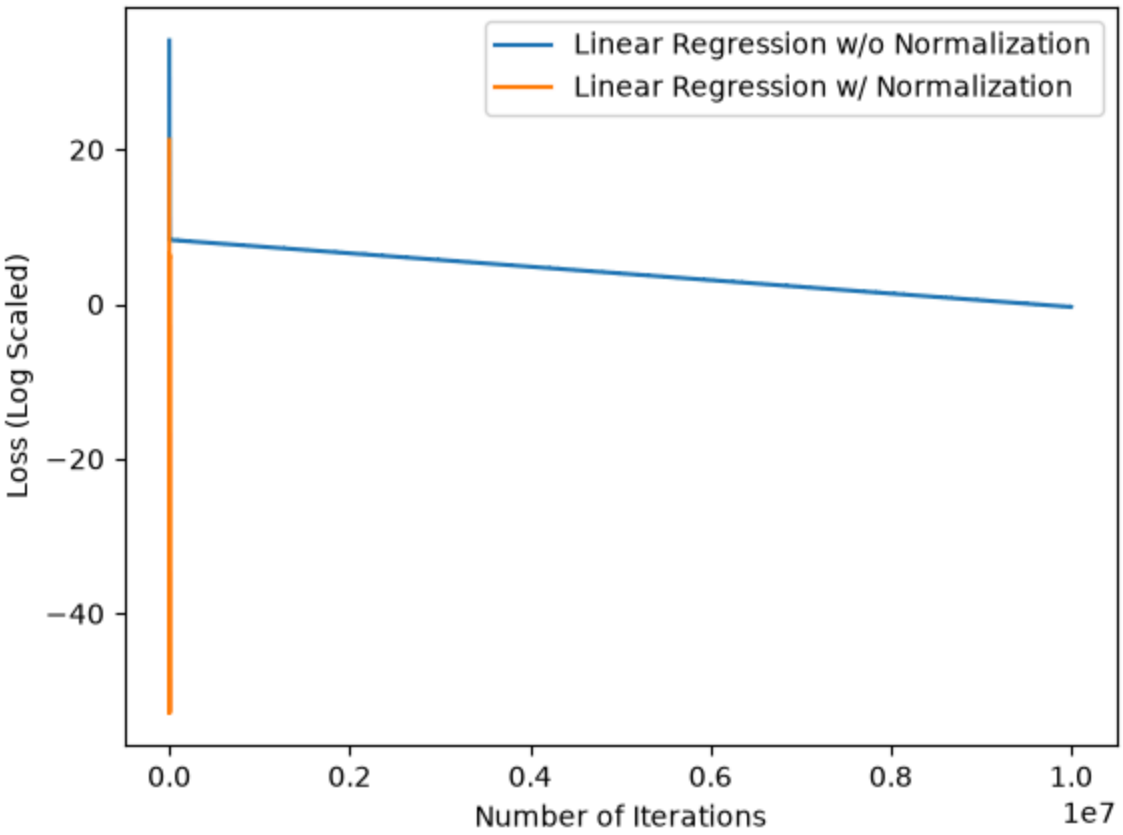
```
Start Training
----------------------
Iteration: 0
Coef: [[3.34606576]
 [5.76641375]]
Loss: 1661161405.4589224
----------------------
End Training (Early Stopped at iteration 1383)

Number of total iterations: 1383
Best Loss: 1.182003936834197e-23
CPU times: total: 62.5 ms
Wall time: 63.7 ms
```

Now, let's compare the performance between these two models with/without normalization

In [16]:
```
plt.plot(np.log(reg.loss), label='Linear Regression w/o Normalization')
plt.plot(np.log(reg_norm.loss), label='Linear Regression w/ Normalization')
plt.xlabel("Number of Iterations")
plt.ylabel("Loss (Log Scaled)")
plt.legend()
plt.show()
```

```
c:\Users\jason\miniconda3\envs\cse190\Lib\site-packages\IPython\core\pylabtools.py:1
58: UserWarning: Creating legend with loc="best" can be slow with large amounts of d
ata.
  fig.canvas.print_figure(bytes_io, **kw)
```



Question 1: What conclusions can you draw from this experiment? Did normalization help? How and why?

- **Answer:** Normalization did help accelerate the convergence speed and improves overall model stability since the raw input X range from 1 to 1000. With normalization, the range of X drastically decreased into the range between 0 and 1, making the scales of all features uniform, which allowed smoother convergence.

Experiment 2: Real-World Data

After you complete the first experiment, let's see how our model performs against real-world data.

The below dataset is taken from the Boston Housing dataset, where there are 13 features and 1 target variable.

0. CRIM - per capita crime rate by town
1. ZN - proportion of residential land zoned for lots over 25,000 sq.ft.
2. INDUS - proportion of non-retail business acres per town.
3. CHAS - Charles River dummy variable (1 if tract bounds river; 0 otherwise)
4. NOX - nitric oxides concentration (parts per 10 million)
5. RM - average number of rooms per dwelling

6. AGE - proportion of owner-occupied units built prior to 1940
7. DIS - weighted distances to five Boston employment centres
8. RAD - index of accessibility to radial highways
9. TAX - full-value property-tax rate per $10,000
10. PTRATIO - pupil-teacher ratio by town
11. B - 1000(Bk - 0.63)^2 where Bk is the proportion of blacks by town
12. LSTAT - % lower status of the population
13. MEDV (**TARGET VARIABLE y**) - Median value of owner-occupied homes in $1000's

In [17]:
```
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/housing/housing.da
df = pd.read_csv(url, delimiter=r'\s+', header=None)
df.head()
```

Out[17]:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

In [18]:
```
X, y = np.array(df.drop(13, axis=1)), np.array(df[13])
```

In [19]:
```
X.min(axis = 0)
```

Out[19]:
```
array([6.3200e-03, 0.0000e+00, 4.6000e-01, 0.0000e+00, 3.8500e-01,
       3.5610e+00, 2.9000e+00, 1.1296e+00, 1.0000e+00, 1.8700e+02,
       1.2600e+01, 3.2000e-01, 1.7300e+00])
```

Now, let's use the data to fit our model

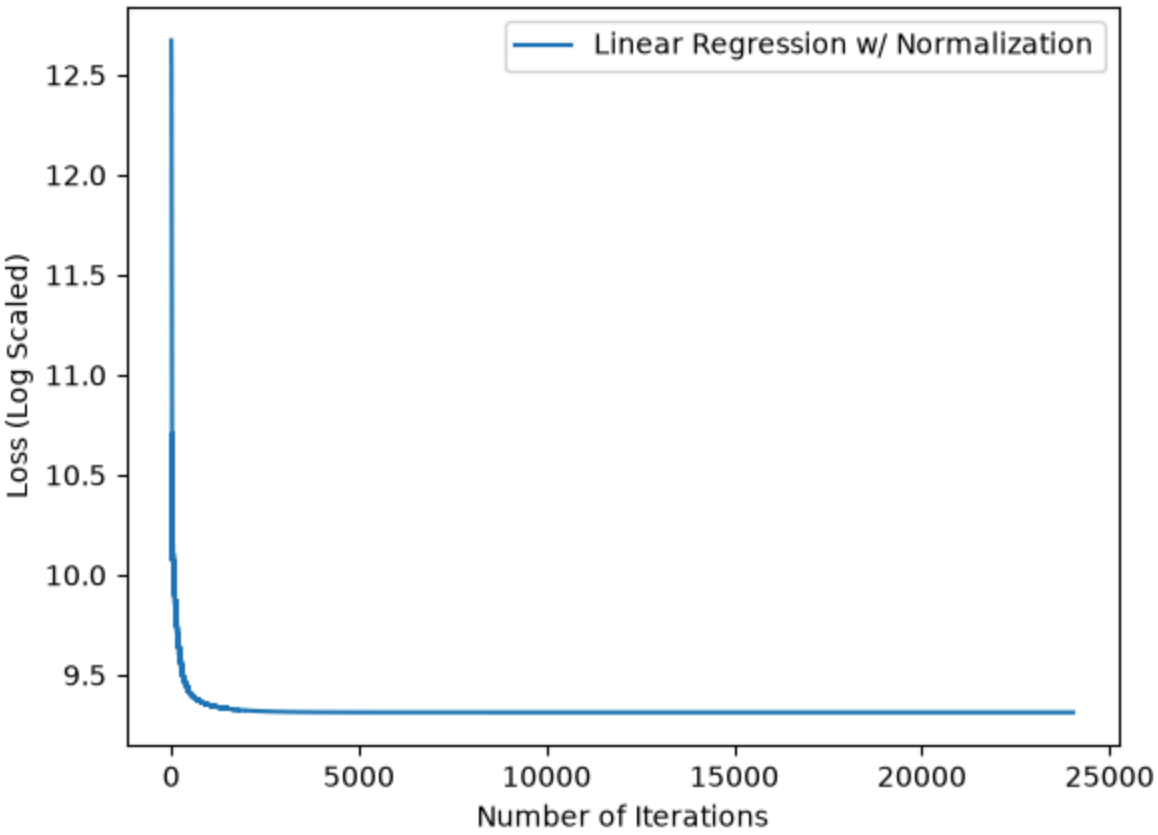
In [20]:
```
%%time
reg = Linear_Regression(num_iter=100000, normalize=True)
reg.fit(X,y)
print(f'\nNumber of total iterations: {len(reg.loss)} \nBest Loss: {min(reg.loss)}'
```

```
Start Training
-----------------------
Iteration: 0
Coef: [[-0.94622938]
 [ 0.29879545]
 [-0.62328578]
 [-0.24962219]
 [-0.58353254]
 [-0.61728497]
 [ 0.5120234 ]
 [ 0.5259475 ]
 [-0.69693873]
 [-0.1872828 ]
 [-0.59354851]
 [ 0.16063365]
 [ 0.97600368]
 [-0.29573891]]
Loss: 318802.0911081078
-----------------------
End Training (Early Stopped at iteration 24031)

Number of total iterations: 24031
Best Loss: 11078.784577955437
CPU times: total: 1.08 s
Wall time: 1.35 s
```

Let's visualize the loss curve of our model on this dataset

In [21]:
```
plt.plot(np.log(reg.loss), label='Linear Regression w/ Normalization')
plt.xlabel("Number of Iterations")
plt.ylabel("Loss (Log Scaled)")
plt.legend()
plt.show()
```



To verify our model, we can compare our model's performance with respect to the linear regression model implemented in scikit-learn (a.k.a. `sklearn`). Scikit-learn is a popular machine learning library in python that provides many classical machine learning algorithms for many different tasks (regression, classification, clustering, etc). It also contains utility functions for preprocessing, calculating metrics, etc.

If you implemented your model correctly, you should get a very similar output (difference < 1e-3) for RMSE (Root Mean Squared Error) compared to sklearn linear regressor's RMSE.

In [23]:
```
m, n = df.shape
X_norm = X.copy()

# TODO: normalize X using the procedure in your model implementation
X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

# Let's build a model with sklearn
lr = LinearRegression()
lr.fit(X_norm,y)

#Compare Root Mean Squared Error.
print(f"Our Model's RMSE: {(sum((reg.predict(X).flatten() - y)**2)/m)**0.5}\
\nSklearn Model's RMSE: {(sum((lr.predict(X_norm) - y)**2)/m)**0.5}")
```

```
Our Model's RMSE: 4.6791912956973745
Sklearn Model's RMSE: 4.679191295697283
```

Now, let's have some tweaks with our custom model. First, let's see if an interception (i.e. bias) really helps with our model's performance on the real-world data.

In [24]:
```
%%time
%%capture
reg_bias = Linear_Regression(num_iter=100000, normalize=True, intercept=True)
reg_no_bias = Linear_Regression(num_iter=100000, normalize=True, intercept=False)
reg_bias.fit(X,y)
reg_no_bias.fit(X,y)
```

```
CPU times: total: 1.69 s
Wall time: 1.8 s
```

In [25]:
```
print(f"Our Model's RMSE with Interception: {(sum((reg_bias.predict(X).flatten() -
\nOur Model's RMSE without Interception: {(sum((reg_no_bias.predict(X).flatten() -
```

```
Our Model's RMSE with Interception: 4.679191295697381
Our Model's RMSE without Interception: 5.2413542310052765
```

Question 2: What conclusions can you make here? Does the addition of an intercept make our model perform better?

- **Answer:** Yes, the model trained with intercep achieved a lower RMSE than the model without (4.67 < 5.24). As such, the addition of an intercept term does improve the model's predictive accuracy, resulting in a lower RMSE on real-world data.

Second, let's see if regularization can further help with decreasing our model's loss. Since regularization deals with the problem of overfitting, we need to check our model's performance on the "unseen" data. Here, we will split our data into two parts: `training set` and `test set`, where our model will be fit with the training set, and the performance will be evaluated based on the test set.

In [26]:
```
X_train, X_test, y_train, y_test = train_test_split(X_norm, y, test_size=0.33, rand
m, n = X_test.shape
```

In [27]:
```
%%time
%%capture
reg = Linear_Regression(num_iter=100000, normalize=True)
reg.fit(X_train, y_train)
# Feel free to tune the lambda hyperparameter for better performance when penalty (
reg_l1 = Linear_Regression(num_iter=100000, normalize=True, penalty='l1')
reg_l1.fit(X_train, y_train)
reg_l2 = Linear_Regression(num_iter=100000, normalize=True, penalty='l2')
reg_l2.fit(X_train, y_train)
```

```
CPU times: total: 1.52 s
Wall time: 1.7 s
```

In [28]:
```
print(f"Our Model's RMSE: {(sum((reg.predict(X_test).flatten() - y_test)**2)/m)**0.
print(f"Our L1 Regularized Model's RMSE: {(sum((reg_l1.predict(X_test).flatten() -
print(f"Our L2 Regularized Model's RMSE: {(sum((reg_l2.predict(X_test).flatten() -
```

```
Our Model's RMSE: 4.5523645522772425
Our L1 Regularized Model's RMSE: 4.551896004391352
Our L2 Regularized Model's RMSE: 4.684665051318574
```

Question 3: What conclusions can you make here? Does the addition of a regularization make our model perform better on the test set? Why does the addition of it make our model perform better/worse?

- **Answer:** For L1, the regularization barely improved; whereas L2 actually made the RMSE higher. This could be caused by the untuned hyperparameter lambda, which introduced some underfitting.

Finally, let's see the role of an adaptive learning rate. Let's see our model's performance when adaptive learning rate is disabled.

In [29]:
```
m, n = X.shape
```

In [30]:
```
%%time
%%capture
reg = Linear_Regression(num_iter=100000, normalize=True)
reg.fit(X, y)
reg_alt = Linear_Regression(num_iter=100000, normalize=True, adaptive=False)
reg_alt.fit(X, y)
```

```
CPU times: total: 5.45 s
Wall time: 6.05 s
```

In [31]:
```
print(f"Our Model's RMSE with Adaptive LR: {(sum((reg.predict(X).flatten() - y)**2)
\nOur Model's RMSE without Adaptive LR: {(sum((reg_alt.predict(X).flatten() - y)**2
```

```
Our Model's RMSE with Adaptive LR: 4.67919129569738
Our Model's RMSE without Adaptive LR: 4.732365352474305
```

Question 4: What conclusions can you make here? Does the addition of an adaptive learning rate make our model perform better? What are your reasonings here?

- **Answer:** The adaptive learning rate does make the model perform better. With step size multiplier of 1.3 for flatter surface and multiplier of 0.9 for steeper curve, it accelerates with converging the model's error towards local minimum.